Unit-2: Data Preprocessing & Supervised Learning

Overview

This unit introduces the essential concepts of data preprocessing and supervised learning techniques in machine learning. It begins with data preprocessing, highlighting the importance of preparing raw data for analysis and covering methods such as mean removal, scaling, normalization, binarization, and label encoding.

The regression section explores linear regression, decision tree regression, and support vector regression (SVR), along with techniques for evaluating models using metrics like MSE, RMSE, and R^2, and discusses model deployment.

The classification section focuses on building simple classifiers, with detailed coverage of logistic regression and naïve Bayes classifiers, while emphasizing the importance of dividing data into training and testing sets, performing cross-validation, measuring accuracy, and interpreting results using confusion matrices and performance reports.

The final part of the unit covers predictive modeling with Support Vector Machines (SVM), explaining how to construct both linear and nonlinear classifiers, extract confidence measurements, and

Pre-processing data

- Pre-processing data involves transforming raw data into a suitable format for machine learning models.
- It includes techniques such as mean removal to center the data, scaling to bring features to a uniform range, normalization to adjust values for consistent magnitude, binarization to convert numerical features into binary form, and label encoding to convert categorical labels into numeric values.
- These steps ensure that the data is clean, consistent, and ready for effective model training and evaluation.
- Data preprocessing can significantly impact the performance and accuracy of machine learning models.

1. **Mean removal**
   - It is a preprocessing technique in which the average value of each feature is subtracted from the dataset so that the transformed data has a mean of zero.
   - This centers the data around the origin, reducing bias due to large feature values and helping many machine learning algorithms, such as regression and SVM, to perform more effectively.

     Example:

     Suppose we have a dataset with one feature (column):

     $X = [2,4,6,8,10]$

     Step 1: Calculate the mean:

     $$\text{Mean} = \frac{2+4+6+8+10}{5} = 6$$

     Step 2: Subtract the mean from each value:

     $X' = [2-6, 4-6, 6-6, 8-6, 10-6]$

     Result after mean removal

     $X' = [-4, -2, 0, 2, 4]$

Now, the new dataset has a mean of 0, which is centered around the origin.

Advantages of Mean Removal

- Centres the data: Shifts features so their mean is zero, making data more balanced.
- Improves numerical stability: Prevents very large or skewed values from dominating calculations.
- Helps optimization: Gradient descent and other optimization algorithms converge faster.
- Improves model performance: Many ML models (e.g., linear regression, PCA, SVM) work better when features are centred.
- Reduces bias: Ensures that feature scaling is not influenced by offset values in data.

```
#Mean removal
import numpy as np
from sklearn import preprocessing
data=np.array([[2],[4],[6],[8],[10]])
#Calculate the mean of each column.
mean=data.mean(axis=0)#axis=0 means, Calculate mean column wise
#Subtract mean from the data
mean_removed=data-mean
print(mean_removed)
print(mean_removed.std(axis=0))#calculates standard deviation of the column.
print(mean_removed.mean(axis=0))#verifies the mean after transformation, Confirms that mean removal worked.
#Scale(), to standardize the data. It means it transorms the data to have a mean of 0 and standard deviation of 1.
data_standardize=preprocessing.scale(data)#does standardization, It subtracts the mean and divides by standard deviation.
print(data_standardize)
print(data_standardize.std(axis=0))
print(data_standardize.mean(axis=0))
```

```
[[-4.]
 [-2.]
 [ 0.]
 [ 2.]
 [ 4.]]
[2.82842712]
[0.]
[[-1.41421356]
 [-0.70710678]
 [ 0.        ]
 [ 0.70710678]
 [ 1.41421356]]
[1.]
[0.]
```

2. **Scaling**
   - Scaling is a fundamental preprocessing step in machine learning aimed at ensuring that numerical features have a similar scale.
   - This is necessary because many machine learning algorithms perform better or converge faster when the input numerical features are on a similar scale.
   - Min-Max Scaling: The minimum value of each feature is set to 0 and the maximum value is set to 1.
   - Scaling brings all values into a fixed range, making features comparable.

Advantages of Scaling:

- Improves algorithm performance: Scaling helps algorithms converge faster and perform better by ensuring that numerical features are on a similar scale.
- Helps model understand the problem: Scaling brings data points closer together, enabling the model to learn and understand the problem.

- Improves separability of classes: Scaling features before reducing dimensionality can improve the separability of classes, which can improve the overall model's performance.
- Below is the formula for scaling:

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- If the value is: ([[10], [20], [30], [40], [50]])
  The calculation would be as follows:
  Xmin=10, Xmax=50
  (10-10) / (50-10) = 0
  (20-10) / (50-10) = 0.25
  (30-10) / (50-10) = 0.5
  (40-10) / (50-10) = 0.75
  (50-10) / (50-10) = 1

```
#Scaling
import numpy as np
from sklearn import preprocessing
data=np.array([[10], [20], [30], [40], [50]])
print(data)
print("*"*70)
data_scaler=preprocessing.MinMaxScaler(feature_range=(0,1))#Creates min-max scaler object, feature_range=(0,1)
scaled_data=data_scaler.fit_transform(data) #Fit, Calculates the minimum and maximum of the data.
                                            #Transform, Applies the min-max scaling formula to each value.
print(scaled_data)
```

```
[[10]
 [20]
 [30]
 [40]
 [50]]
**********************************************************************
[[0.  ]
 [0.25]
 [0.5 ]
 [0.75]
 [1.  ]]
```

Normalization

- Data Normalization is very important pre-processing step in Machine Learning (ML) that makes a difference to make sure that all input parameters are scaled to a common range.
- Data normalization in ML transforms data into a common format, so it can be used in analytics and machine learning algorithms.
- It is typically used to transform raw data into a more useful form for ML algorithms such as linear regression, logistic regression, and neural networks.
- Data Normalization in machine learning can be applied to numerical and categorical data, and it can help to reduce the complexity of the data.

Advantages of Normalization:

- To remove the impact of scale
- To improve the performance of models
- To address skewness in the data
- To improve the interpretability of the data

Example: (Here L stands for lebesgue space)

data = np.array([[3, 4], [1, 2], [5, 12]])

The formula for L1 normalization is:

$$X' = \frac{X}{\sum |X|}$$

Let us take the data of 1st row, [3, 4]:

[3/7, 4/7] = [0.428, 0.571]

The formula for L2 normalization is:

$$X' = \frac{X}{\sqrt{\sum X^2}}$$

Let us take the data of 1st row, [3, 4]:

$3^2+4^2=9+16=25$, then root of 25. It means 5.

[3/5,4/5] = [0.6,0.8]

```python
#Normalization
import numpy as np
from sklearn import preprocessing

# Multi-feature dataset (2 features per row)
data = np.array([[3, 4],
                 [1, 2],
                 [5, 12]])

print("Original Data:\n", data)
print("*" * 70)

# --- L1 Normalization (sum of absolute values = 1) ---
normalized_l1 = preprocessing.normalize(data, norm="l1")
print("After L1 Normalization:\n", normalized_l1)

# --- L2 Normalization (Euclidean length = 1) ---
normalized_l2 = preprocessing.normalize(data, norm="l2")
print("\nAfter L2 Normalization:\n", normalized_l2)
```

```
Original Data:
 [[ 3  4]
 [ 1  2]
 [ 5 12]]
**********************************
After L1 Normalization:
 [[0.42857143 0.57142857]
 [0.33333333 0.66666667]
 [0.29411765 0.70588235]]

After L2 Normalization:
 [[0.6        0.8       ]
 [0.4472136  0.89442719]
 [0.38461538 0.92307692]]
```

Binarization

- Binarization is a technique used in machine learning and image processing to convert data into binary values, typically 0s and 1s.
- Binarization is used to convert numerical data into binary values by setting a threshold value.
- If the value is greater than the threshold, it becomes 1.
- If a value is less than or equal to the threshold, it becomes 0.
- For example, if you set a threshold of 8,00,000 for annual income, any income value less than or equal to 8,00,000 would be assigned 0, and any income value greater than 8,00,000 would be assigned 1.
- This is useful when we want to highlight the presence or absence of a feature, rather than its actual value.

Advantages of Binarization

- Converts continuous values into 0/1 values.
- Useful when only the presence/absence matters.
- Very large or very small values are treated equally.
- Word occurrence (present = 1, absent = 0).

```
#Binarization
import numpy as np
from sklearn import preprocessing
data=np.array([[10.2], [-22.5], [30], [-14.22], [50]])
print(data)
print("*"*70)
binarized_data=preprocessing.Binarizer(threshold=-10).transform(data)
print(binarized_data)
```

```
[[ 10.2 ]
 [-22.5 ]
 [ 30.  ]
 [-14.22]
 [ 50.  ]]
**********************************************************************
[[1.]
 [0.]
 [1.]
 [0.]
 [1.]]
```

Label Encoding

- Label encoding is a technique used in machine learning to convert categorical data into numerical values so that machine learning algorithms can process it.
- Label encoding assigns a unique integer to each category in a categorical variable.
- For example, if a categorical variable has the categories “Mango”, “Banana”, “Orange”, and “Apple”, the labels 0, 1, 2 and 3 could be assigned to each category, respectively.
- It is particularly useful when working with algorithms that require numerical input, as most machine learning models can only operate on numerical data.

Advantages of Label Encoding

- Easy to implement.
- Replace text with small integers.
- Makes categorical features usable in ML models.

```python
#Label encoding
from sklearn import preprocessing
# Data
labels = ['apple', 'mango', 'banana', 'apple', 'grapes', 'mango', 'grapes']
print("Labels: ", labels)
# Load the Encoder
label_encoder = preprocessing.LabelEncoder()
# Fit and transform
op = label_encoder.fit_transform(labels)
print("Encoded Labels: ", op)
print("="*70)
# Test the encoder by applying the new labels
test_labels = ['mango', 'banana']
print("Test Labels: ", test_labels)
print("Test labels encoded: ", label_encoder.transform(test_labels))
print("="*70)
# Inverse transform
test_code = [2, 0]
print("Testing codes: ", test_code)
print("Testing labels: ", label_encoder.inverse_transform(test_code))
```

```
Labels:  ['apple', 'mango', 'banana', 'apple', 'grapes', 'mango', 'grapes']
Encoded Labels:  [0 3 1 0 2 3 2]
======================================================================
Test Labels:  ['mango', 'banana']
Test labels encoded:  [3 1]
======================================================================
Testing codes:  [2, 0]
Testing labels:  ['grapes' 'apple']
```

What is Regression?

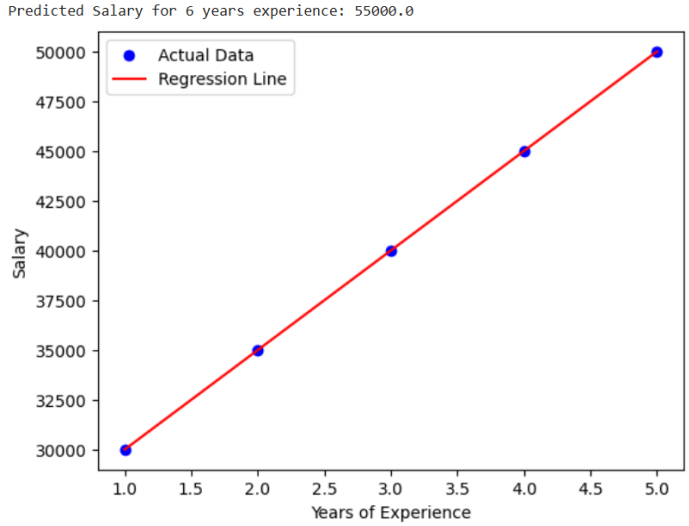
- Regression is a supervised machine learning technique.
- It is used when the output (dependent variable) is continuous (not categorical).
- The main idea is to find a relationship between input variables (features) and the output variable.
- The key objective is to build a mathematical model that can predict unknown outputs based on known inputs.
- Example:
    - Predicting house price from size, location, and age.
    - Predicting stock price from past trends.
    - Predicting sales from advertising spend.
- Foundation for more advanced algorithms like logistic regression, neural networks, etc.

Linear Regression

- Linear Regression is a supervised learning algorithm used to predict a continuous output by finding a linear relationship between input variable(s) and output.
- It assumes that the change in the output is proportional to the change in input(s).

```
#Linear regression
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Training data
X = np.array([[1], [2], [3], [4], [5]])   # Feature
y = np.array([30000, 35000, 40000, 45000, 50000])  # Target

# Train model
model = LinearRegression()
model.fit(X, y)

# Predictions
predicted_salary = model.predict([[6]])

print("Predicted Salary for 6 years experience:", predicted_salary[0])

# Visualization
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(X, model.predict(X), color="red", label="Regression Line")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.legend()
plt.show()
```

```
Predicted Salary for 6 years experience: 55000.0
```



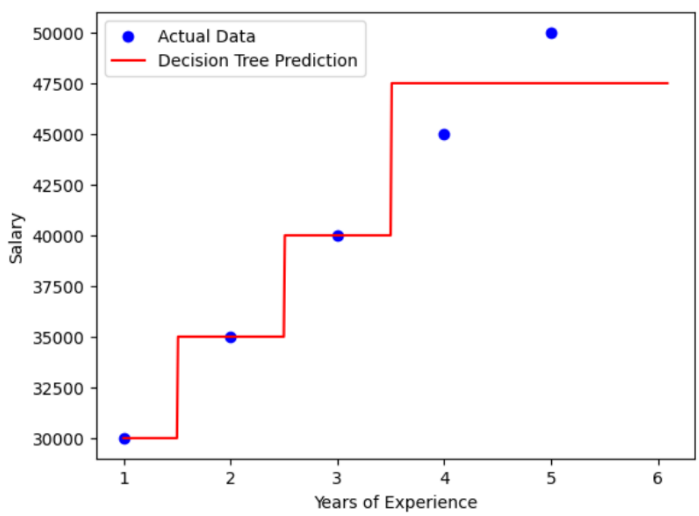
Decision Tree Regression

- A Decision Tree regression model predicts continuous values by splitting data into regions using decision rules (if–else conditions).
- Instead of fitting a straight line like Linear Regression, it divides the input space into intervals and predicts the average value in each region.
- Start with the full dataset.
- Split the data based on a condition (e.g., “Is experience < 3 years?”).

- Can model non-linear relationships.

```python
#Decision tree
import numpy as np
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt

# Dataset
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([30000, 35000, 40000, 45000, 50000])

# Train Decision Tree Regressor
model = DecisionTreeRegressor(max_depth=2)
model.fit(X, y)

# Predict
print("Prediction for 6 years experience:", model.predict([[6]])[0])

# Visualization
X_test = np.arange(1, 6.1, 0.01).reshape(-1,1)
y_pred = model.predict(X_test)
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(X_test, y_pred, color="red", label="Decision Tree Prediction")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.legend()
plt.show()
```

```
Prediction for 6 years experience: 47500.0
```

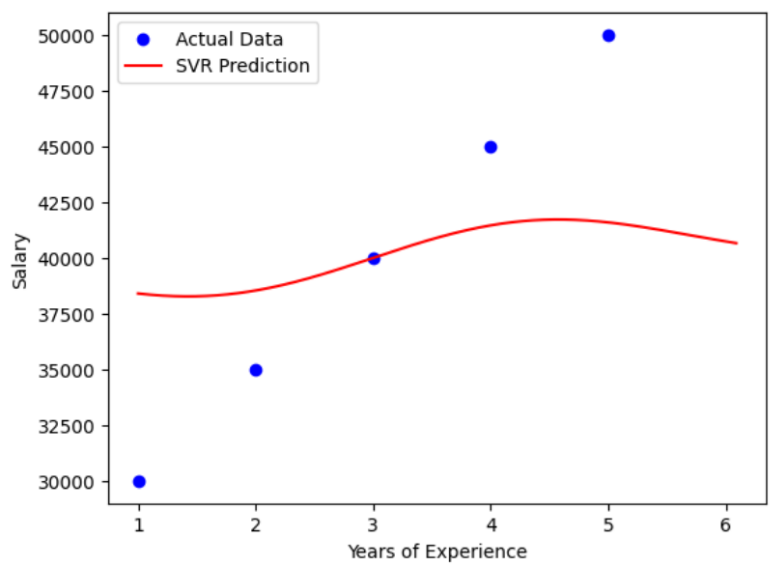


Support Vector Machines (SVM)

- Instead of fitting a line to all points, SVR tries to fit a line (or curve) that keeps most data points within a margin.
- It allows some tolerance for error but keeps predictions as flat (simple) as possible.
- Predictions inside this margin are considered "good enough" (no penalty).
- Works well with non-linear relationships.
- Computationally expensive for large datasets.
- Uses the Radial Basis Function kernel
- This lets the model capture non-linear relationships between input (X) and output (y).
- If the relationship between features (like house size, age, distance) and price is curved or complex, RBF works better than a straight line.

```python
#Support Vector Machine (SVR is a part of SVM)
import numpy as np
from sklearn.svm import SVR  #Scikit-learn
import matplotlib.pyplot as plt

# Dataset
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([30000, 35000, 40000, 45000, 50000])

# Train Support Vector Regressor with RBF kernel
model = SVR(kernel='rbf', C=1000, epsilon=500) #rbf=Radial Basis Function kernel
                                               #c=Regularization parameter. Higer the value of c, means works hard to minimize error and vice versa
                                               #epsilon=500, Errors within ±500 are ignored (no penalty).
model.fit(X, y)

# Predict
print("Prediction for 6 years experience:", model.predict([[6]])[0])

# Visualization
X_test = np.arange(1, 6.1, 0.01).reshape(-1,1)
y_pred = model.predict(X_test)
plt.scatter(X, y, color="blue", label="Actual Data")
plt.plot(X_test, y_pred, color="red", label="SVR Prediction")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.legend()
plt.show()
```

```
Prediction for 6 years experience: 40741.52675366817
```



Classification

- Labeled training data are given to the classifier.
- As training data has labeled or category defined the task of a classifier is to map testing dataset and assign some labeled or category.
- Example:
  - In a banking domain, to identify fraudulent transactions, millions of transactions need to be scrutinized.
- It is not possible for any human being to carry out this task.

```python
#Classifier
#Used simple classifier without ML algorithms
# Simple Classifier Example: Pass or Fail
import numpy as np

# Step 1: Create data (students' marks)
marks = np.array([25, 35, 40, 55, 65, 75, 90])

# Step 2: Define rule-based classifier
def simple_classifier(score):
    if score >= 40:
        return "Pass"
    else:
        return "Fail"

# Step 3: Apply classifier to all marks
results = [simple_classifier(m) for m in marks]

# Step 4: Display results
for m, r in zip(marks, results):
    print(f"Marks: {m} → Result: {r}")
```

```
Marks: 25 → Result: Fail
Marks: 35 → Result: Fail
Marks: 40 → Result: Pass
Marks: 55 → Result: Pass
Marks: 65 → Result: Pass
Marks: 75 → Result: Pass
Marks: 90 → Result: Pass
```

Logistic Regression

- Despite the name, Logistic Regression is a Classification Algorithm (not regression).
- It is used when the target variable is categorical (e.g., Pass/Fail, Yes/No, Spam/Not Spam).
- Instead of predicting continuous values (like Linear Regression), it predicts probabilities between 0 and 1.

```python
#Classifier using Logistic regression, an algorithm of ML
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Create dataset
data = {
    'Marks': [25, 35, 40, 55, 65, 75, 90],
    'Result': ['Fail', 'Fail', 'Pass', 'Pass', 'Pass', 'Pass', 'Pass']
}
df = pd.DataFrame(data) #creating the data frame from the data

# Convert labels (Pass=1, Fail=0)
df['Result'] = df['Result'].map({'Fail': 0, 'Pass': 1}) #Because ML is more convenient with numbers

X = df[['Marks']]   # input
y = df['Result']    # output

# Train-test split (70% of data for training and 30% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1) # random_state=1, ensures the split is the same every time

# Train Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Test with new data
new_marks = np.array([[30], [45], [80]])
predictions = model.predict(new_marks)
print("\nPredictions for new students:")
for m, p in zip(new_marks, predictions):
    print(f"Marks: {m[0]} → {'Pass' if p==1 else 'Fail'}")
```

```
Predictions for new students:
Marks: 30 → Fail
Marks: 45 → Pass
Marks: 80 → Pass
```

Naive Bayes algorithm

- Naive Bayes works on Bayes' theorem.
- It assumes that all features are independent.
- Works well for text classification, spam filtering, sentiment analysis, etc.
- Naive Bayes algorithm is fast and efficient on large datasets.
- Works surprisingly well even with the independence assumption.
- Best for text and categorical data.

```python
#Naive Bayes Algorithm
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Sample dataset
data = {
    'Message': [
        "Win money now",
        "Lowest price offer",
        "Hello friend how are you",
        "Let's meet tomorrow",
        "Congratulations, you won a lottery",
        "Reminder for meeting at 5 PM"
    ],
    'Label': ['Spam', 'Spam', 'Not Spam', 'Not Spam', 'Spam', 'Not Spam']
}
df = pd.DataFrame(data)
# Convert labels to numeric
df['Label'] = df['Label'].map({'Not Spam': 0, 'Spam': 1})
X = df['Message']   # Text data
y = df['Label']     # Target

# Convert text into numerical features
vectorizer = CountVectorizer() #Converts text into numbers
X_features = vectorizer.fit_transform(X) # fit learns all unique words,
                                         # transorm converts each sentence into a vector of word counts.

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_features, y, test_size=0.3, random_state=1)
# Train Naïve Bayes classifier
model = MultinomialNB()
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)

# Test with new emails
new_emails = ["Free offer just for you", "Hi, are we still on for today?"]
new_features = vectorizer.transform(new_emails)
predictions = model.predict(new_features)

print("\nPredictions for new emails:")
for email, p in zip(new_emails, predictions):
    print(f"Email: '{email}' → {'Spam' if p==1 else 'Not Spam'}")
```

```
Predictions for new emails:
Email: 'Free offer just for you' → Spam
Email: 'Hi, are we still on for today?' → Not Spam
```

Regression Algorithms vs. Classification Algorithms

Regression Algorithms	Classification Algorithms
The output variable must be either continuous nature or real value.	The output variable must be a discrete value.
The regression algorithm's task is mapping input value (x) with continuous output variable (y).	The classification algorithm's task is mapping the input value of x with the discrete output variable of y.
They are used with continuous data.	They are used with discrete data.
It attempts to find the best fit line, which predicts the output more accurately.	Classification tries to find the decision boundary, which divides the dataset into different classes.
Regression algorithms solve regression problems such as house price predictions and weather predictions.	Classification algorithms solve classification problems like identifying spam emails, spotting cancer cells, and speech recognition.
We can further divide Regression algorithms into Linear and Non-linear Regression.	We can further divide Classification algorithms into Binary Classifiers and Multi-class Classifiers.

Training and Testing data set

- In machine learning, the main goal is to develop a model that can generalize well that means it should not just memorize the data it has already seen but should also perform well on new, unseen data.
- To achieve this, we divide the available dataset into at least two parts:
  - a training dataset and
  - a testing dataset.

Training Dataset

- The training dataset is the portion of the data used by the algorithm to learn patterns, relationships, and rules.
- For example, in a dataset of student marks and pass/fail labels, the training dataset will help the model understand how marks relate to the outcome.
- During training, the algorithm adjusts its internal parameters so that predictions on the training data are as accurate as possible.

Testing Dataset

- The testing dataset is kept separate and is not shown to the model during training.
- It acts as a proxy for new data and is used to evaluate the model after training is complete.
- Testing answers the question: "How well will my model perform in the real world?"
- For instance, if a student's marks weren't part of the training set, the testing dataset will check if the model can still predict pass/fail correctly.
- By keeping testing data separate, we get a fair and unbiased measure of the model's true performance.

Typical Split Ratios

- Common splits are:
  - **70% Training : 30% Testing**
  - **80% Training : 20% Testing**
- In some cases (like very large datasets), even **90/10 split** is enough.

Accuracy using Cross Validation

- When we train a machine learning model, we usually split the dataset into training and testing sets (70:30 or 80:20).
- However, this approach has a limitation that the accuracy depends heavily on how the data is split.
- If by chance the test set is too easy or too hard, the reported accuracy may be misleading.
- To solve this, we use Cross-Validation.
- Cross-validation is a technique to evaluate machine learning models by splitting the dataset into multiple parts (called folds) and ensuring every data point gets a chance to be in the test set.
- The most common method is k-Fold Cross-Validation:
    - The dataset is split into k equal parts (folds).
    - The model is trained on k-1 folds and tested on the remaining fold.
    - This process is repeated k times, each time with a different fold as the test set.
    - The results (accuracies) from each fold are averaged to give a more reliable accuracy score.

Benefits of Cross Validation

- More reliable accuracy: Uses multiple train-test splits.
- Reduces bias: Every sample is used for both training and testing.
- Good for small datasets: Ensures no data is wasted.

Confusion Matrix

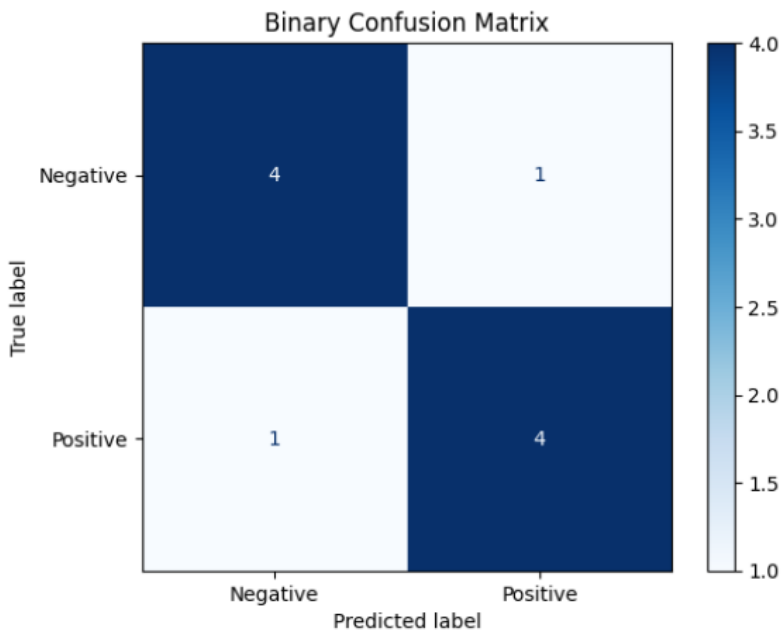
- A confusion matrix is a table that summarizes how well a classification model performs.
- It is considered as a performance measurement tool for classification problems.
- It compares the model's predicted labels against the actual labels.
- It is a table that compares the actual class labels with the predicted class labels given by the model.
- For a binary classification problem (2 classes → Positive, Negative), the matrix looks like this:

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Terms

- **True Positive (TP):** Correctly predicted positives.
- **True Negative (TN):** Correctly predicted negatives.
- **False Positive (FP):** Incorrectly predicted positive (Type I Error).
- **False Negative (FN):** Incorrectly predicted negative (Type II Error).

```python
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Example: true vs predicted labels (binary: 0=Negative, 1=Positive)
y_true = [0, 0, 1, 1, 0, 1, 0, 1, 1, 0]
y_pred = [0, 1, 1, 1, 0, 0, 0, 1, 1, 0]

# Generate confusion matrix
cm = confusion_matrix(y_true, y_pred)

# Display confusion matrix with labels
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Negative", "Positive"])
disp.plot(cmap=plt.cm.Blues, values_format='d')#d means integer

plt.title("Binary Confusion Matrix")
plt.show()
```



Extracting Performance Report

- Once a classification model is trained, simply looking at accuracy is not enough to understand how well the model performs.
- Accuracy only tells us what percentage of total predictions are correct, but it does not explain how the model behaves on each class or how it deals with imbalanced data.
- To solve this, machine learning provides a Performance Report (also called a Classification Report).
- It includes a set of metrics such as Precision, Recall, F1-score, Support, and Accuracy.
- These metrics give a deeper understanding of the strengths and weaknesses of a model.

```python
#Extracting performance report
#We would check the performance of the model, build with Logistic regression.
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Create dataset
data = {
    'Marks': [25, 35, 40, 55, 65, 75, 90],
    'Result': ['Fail', 'Fail', 'Pass', 'Pass', 'Pass', 'Pass', 'Pass']
}
df = pd.DataFrame(data) #creating the data frame from the data

# Convert labels (Pass=1, Fail=0)
df['Result'] = df['Result'].map({'Fail': 0, 'Pass': 1}) #Because ML is more convenient with numbers

X = df[['Marks']]   # input
y = df['Result']    # output

# Train-test split (70% of data for training and 30% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1) # random_state=1, ensures the split is the same every time

# Train Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Test with new data
new_marks = np.array([[30], [45], [80]])
predictions = model.predict(new_marks)
print("\nPredictions for new students:")
for m, p in zip(new_marks, predictions):
    print(f"Marks: {m[0]} → {'Pass' if p==1 else 'Fail'}")
print(report)
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Building linear and nonlinear classifiers using Support Vector Machine (SVM)

- Support Vector Machine (SVM) is a supervised machine learning algorithm that can be used for classification and regression tasks.
- The goal is to find the best decision boundary (hyperplane) that separates data points of different classes with the maximum margin.

Linear Classifier using SVM

- When the data is linearly separable, SVM tries to find a straight line that separates the classes.
- Imagine classifying emails as "Spam" vs "Not Spam" using only two features, number of suspicious words and email length.
- If a straight line can separate them, then the linear SVM is enough.

Nonlinear Classifier using SVM

- Real-world data is often not linearly separable.
  To handle this, SVM uses the Kernel Trick, mapping data into higher dimensions where it becomes linearly separable.
- Suppose you want to classify points in a circle, inner points = Class A, outer points = Class B.
- A straight line cannot separate them.

- By applying a kernel (like RBF), SVM projects the data into higher dimensions, where a linear separation is possible.
- RBF (Radial Basis Function) Kernel is the most popular kernel that handles complex nonlinear boundaries.

```
#Classifier using linear SVM
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_moons
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Generate linearly separable data
X, y = make_classification(n_samples=200, #total samples
                           n_features=2, #number of features
                           n_informative=2, #informative features
                           n_redundant=0, #no redundant features
                           n_clusters_per_class=1, # 3 output classes
                           random_state=1)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Linear SVM
linear_svm = SVC(kernel='linear')
linear_svm.fit(X_train, y_train)
y_pred_linear = linear_svm.predict(X_test)

print("Linear SVM Report:\n", classification_report(y_test, y_pred_linear))
```

```
Linear SVM Report:
              precision    recall  f1-score   support

           0       0.97      0.83      0.89        35
           1       0.80      0.96      0.87        25

    accuracy                           0.88        60
   macro avg       0.88      0.89      0.88        60
weighted avg       0.90      0.88      0.88        60
```

```python
#Classifier using non-linear SVM
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification, make_moons
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
# Generate nonlinear (moon-shaped) data
X, y = make_moons(n_samples=200, noise=0.2, random_state=1)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

# Nonlinear SVM with RBF kernel
rbf_svm = SVC(kernel='rbf')
rbf_svm.fit(X_train, y_train)
y_pred_rbf = rbf_svm.predict(X_test)

print("Nonlinear (RBF) SVM Report:\n", classification_report(y_test, y_pred_rbf))
```

```
Nonlinear (RBF) SVM Report:
               precision    recall  f1-score   support

           0       0.96      0.96      0.96        27
           1       0.97      0.97      0.97        33

    accuracy                           0.97        60
   macro avg       0.97      0.97      0.97        60
weighted avg       0.97      0.97      0.97        60
```

Extracting Confidence Measurements

What are Confidence Measurements?

- When a machine learning model (like SVM, Logistic Regression, or Naïve Bayes) makes predictions, it doesn't just assign a class, it can also estimate how confident it is about that prediction.
- In other words, a confidence measurement tells us the probability or degree of certainty that the model assigns to each possible class.

Predicted Class	Confidence (Probability)
Class A	0.85 (85%)
Class B	0.10 (10%)
Class C	0.05 (5%)

Why Confidence Measurements Are Important

1. **Better Decision-Making:**
   In real-world systems (medical diagnosis, fraud detection, etc.), we need to know *how sure* the model is before acting.
2. **Error Handling:**
   Low-confidence predictions can be flagged for human review.
3. **Model Evaluation:**
   Confidence helps identify overconfident or underconfident models.

4. **Threshold Adjustment:**
   Instead of always taking the class with the highest probability, we can set a confidence threshold (e.g., accept only predictions >80% sure).

Sample confidence score:

```
[[0.02 0.96 0.02]
 [0.10 0.80 0.10]
 [0.70 0.20 0.10]
 [0.05 0.05 0.90]
 [0.80 0.15 0.05]]
```

Sample 1 → Predicted class: 1, Confidence: 0.96
Sample 2 → Predicted class: 1, Confidence: 0.80
Sample 3 → Predicted class: 0, Confidence: 0.70

What are Performance Metrics?

- Performance metrics are quantitative measures used to evaluate how well a machine learning model performs on unseen data.
- They help us answer questions like:
  - Is the model accurate?
  - Which class does it misclassify most often?
  - How reliable are the predictions?
- Without proper metrics, we cannot judge if our model is truly effective.
- When we build a classifier (like Logistic Regression, Naïve Bayes, or SVM), simply checking how many predictions are correct is not enough.
- For example:
- A model that predicts "Not Spam" for every email might be 95% accurate, but it completely fails to detect actual spam emails.
- Hence, we need detailed metrics that explain *how* and *where* the model performs well or poorly.

Accuracy

- Measures the percentage of correct predictions.

Precision

- Out of all predicted positives, how many are actually positive?

Recall (Sensitivity or True Positive Rate)

- Out of all actual positives, how many did the model correctly identify?

F1-Score

- Harmonic mean of precision and recall.

Confusion Matrix

- It shows how many predictions fell into each true/predicted category.

ROC Curve and AUC

- ROC Curve (Receiver Operating Characteristic): plots True Positive Rate vs False Positive Rate.
- AUC (Area Under Curve): measures how well the model distinguishes between classes.